

FIT3155

SAMPLE EXAM PAPER

FIT3155 SAMPLE EXAM PAPER

Generated Sample Exam 02

INSTRUCTIONS

- This exam is worth 60 marks.

General exam technique

- Some students lose marks by not attempting all questions.
- Suppose you are likely to get 3/6 on a question after spending say 10 minutes on it. Spending another 10 minutes on the same question may gain you at most 3 additional marks. In contrast, spending those 10 minutes on a new question could earn you another 6 marks. The key point is: do not get stuck on one question.
- A good strategy is to first answer the questions you are most confident about before attempting the others. In other words, answering questions strictly serially during an exam can be suboptimal.
- Do not waste time answering an unrelated question in the hope of gaining partial marks. Marking will be based on the question that was actually asked.
- Where necessary, justify your answers with clear and concise explanations, without being overly wordy.

Question 1**(6 marks)**

A *border* of a string $S[1 \dots n]$ is any *proper* prefix of S (i.e., a prefix of length strictly less than n) that is also a suffix of S .

Example: $S[1 \dots 7] = \text{abacaba}$ has exactly two borders, **a** (of length 1) and **aba** (of length 3).

Write pseudocode describing an efficient algorithm that accepts any input string $S[1 \dots n]$ and outputs the lengths of *all* borders of S . State the worst-case time complexity of your algorithm.

(For this problem, you may assume that **zalg** is available as a built-in function that takes a string as input and returns the array of its Z-values. Do not write pseudocode for **zalg**.)

(Page intentionally left blank for working space)

Question 2**(6 marks)**

In the Boyer–Moore algorithm for exact matching of the pattern $pat[1 \dots m]$ against the text $txt[1 \dots n]$, consider a general iteration in which the region $txt[j \dots j + m - 1]$ of the text is being compared with pat , and a right-to-left scan results in a mismatch at position $1 \leq k \leq m$ of the pattern, against some character x in the text.

Let $R(x)$ denote the position of the *rightmost* occurrence of the character x in pat , and suppose that $R(x) < k$. Under the bad character rule, pat is shifted $k - R(x)$ positions to the right, so that the occurrence of x at $pat[R(x)]$ becomes aligned with the mismatched character x in the text.

Prove that this bad character shift is safe, i.e., such a shift can never skip pat past any valid occurrence in txt .

(Page intentionally left blank for working space)

Question 3**(6=3+3 marks)**

Assume throughout that \$ is a unique terminal symbol that is lexicographically smaller than every other character.

- (i) Compute the Burrows–Wheeler Transform of the string

$$S = \text{cocoa\$}.$$

Show the sorted rotations (or sorted suffixes) you used to obtain your answer.

- (ii) The Burrows–Wheeler Transform of some unknown string T \$ is

$$L[1 \dots 6] = \text{abbaa\$}.$$

Recover T . Show your working, e.g., the last-to-first (LF) column mapping you used at each step of the inversion.

(Page intentionally left blank for working space)

Question 4**(6=3+3 marks)**

These questions relate to Ukkonen's linear-time suffix tree construction algorithm for a string $S[1 \dots n]$.

- (i) Suppose suffix extension j of some phase i creates a new leaf numbered j (using Rule 2). Prove that in *every* subsequent phase $i' > i$, suffix extension j is completed by Rule 1 (the 'leaf-edge extension rule'), i.e., "once a leaf, always a leaf".
- (ii) In the *rapid leaf extension* trick, the label of every leaf edge is stored as a pair

$(start, \text{globalEnd}),$

where `globalEnd` is a single mutable integer shared by all leaf edges. Describe how incrementing `globalEnd` implements all Rule 1 extensions of a phase in $O(1)$ total time, and explain why your result from part (i) guarantees that this trick is sound.

(Page intentionally left blank for working space)

Question 5**(6=3+3 marks)**

- (i) Decode the following Elias-Omega codeword to recover the positive integer N that it encodes:

0011101

Show each component you read, and how it determines the length of the next component.

- (ii) Encode the following string into $\langle offset, length, next\ char \rangle$ triples using the LZ77 algorithm:

abcabcabcd

Assume that the search window size and the lookahead buffer size are both 15. List every triple your encoder emits, in order.

(Page intentionally left blank for working space)

Question 6**(6=3+3 marks)**

Let u and v be two n -bit positive integers, where n is a power of two. Write

$$u = 2^{n/2} U_1 + U_0, \quad v = 2^{n/2} V_1 + V_0,$$

where U_1, U_0, V_1, V_0 are $n/2$ -bit integers.

- (i) Show how the product $u \cdot v$ can be computed using only *three* multiplications of $n/2$ -bit integers, together with additions, subtractions, and bit shifts (Karatsuba's method). State the identity you use explicitly.
- (ii) Write down the recurrence describing the running time $T(n)$ of the resulting divide-and-conquer multiplication algorithm, and solve it to obtain a tight asymptotic bound on $T(n)$.

(Page intentionally left blank for working space)

Question 7**(6=3+3 marks)**

A Fibonacci heap H consists of three min-heap-ordered trees, whose roots appear in the root list in the following order (left to right):

- a tree rooted at 4, where node 4 has children 9 and 11, and node 9 has a single child 12;
- a tree rooted at 6, where node 6 has a single child 8;
- a single node 18.

Node 9 is *marked*; no other node is marked. $H.min$ points to node 4.

Use the following conventions: nodes cut from a tree are appended to the right end of the root list in the order they are cut; during consolidation the root list is processed left to right; and when two trees of equal degree are merged, the root with the smaller key becomes the parent.

- Perform DECREASE-KEY($12 \rightarrow 3$) on H . Draw the resulting heap, indicating $H.min$ and any marked nodes.
- On the heap resulting from part (i), perform EXTRACT-MIN. Draw the resulting heap after consolidation, indicating $H.min$ and any marked nodes.

(Page intentionally left blank for working space)

Question 8**(6 marks)**

A k -bit binary counter data structure supports an increment by 1 operation (modulo 2^k), starting with all k bits of the counter initialized to 0.

Using the *potential* method of amortized analysis, determine the amortized time complexity of an increment operation for this data structure.

State your potential function Φ explicitly, verify that it satisfies the required boundary conditions, and show the calculation of the amortized cost of a general increment.

(Page intentionally left blank for working space)

Question 9**(6=3+3 marks)**

Insert the keys

10, 20, 30, 40, 50, 25, 35, 5

in this order into an initially empty B-tree with minimum degree $t = 2$ (i.e., every node holds at most $2t - 1 = 3$ keys).

Use *proactive* (top-down) splitting: while travelling from the root towards the insertion leaf, any full node encountered is split before it is stepped into.

- (i) Draw the B-tree immediately after the insertion of key 40.
- (ii) Draw the final B-tree after all eight insertions. Clearly indicate the keys in every node.

(Page intentionally left blank for working space)

Question 10**(6 marks)**

Solve the following optimization problem using the tableau simplex algorithm. Show the tableau after each pivot, and state the optimal values of x , y , and z .

$$\begin{array}{ll}\text{Maximize} & z = 2x + 3y \\ \text{Subject to} & x + y \leq 4, \\ & x + 3y \leq 6, \\ & x \geq 0, \quad y \geq 0.\end{array}$$

(Page intentionally left blank for working space)

End of Exam